

Project 2 Report

pWordcount: A Pipe-based WordCount Tool

Maha Zainab

COMP 7500

February 5, 2026

1 Introduction

In this project, I implemented a Unix pipe-based word counting program called **pword-count**. As the goal of this assignment was to understand how two separate processes can communicate using ordinary Unix pipes.

Through this project, I learned how pipes work at a low level, how to use `fork()`, and how to carefully manage file descriptors to avoid errors or deadlocks.

2 System Design

2.1 Two Cooperating Processes

I implemented two processes and detail of them is given below.

Process 1 (Parent)

- Opens and reads the input file.
- Sends file content to Process 2 via Pipe #1.
- Receives the word count result from Process 2 via Pipe #2.
- Prints the final word count.

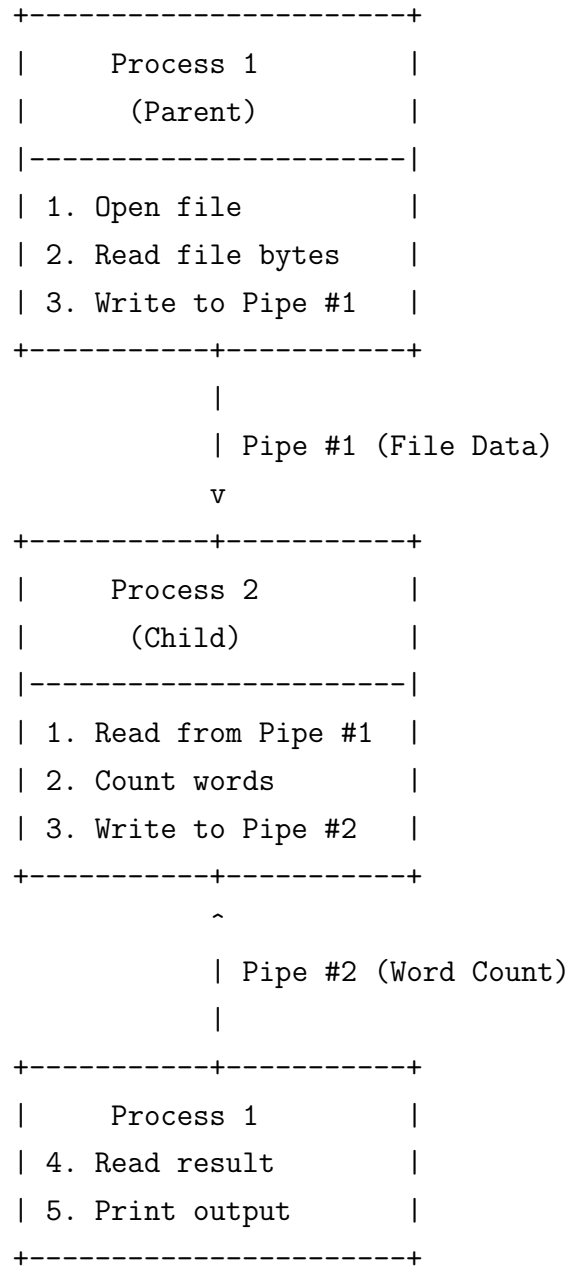
Process 2 (Child)

- Receives file content from Pipe #1.
- Counts the number of words.
- Sends the integer result back to Process 1 via Pipe #2.

Two pipes are required:

- Pipe #1: Parent $\rightarrow$ Child (file data)
- Pipe #2: Child $\rightarrow$ Parent (word count result)

3 Data Flow Diagram



4 Important Functions

To keep the program organized, I used separate compilation. The word counting logic is implemented in a separate file from the pipe logic.

Function Prototype from wordcount.h

```
int count_words_in_buffer(const unsigned char *buf,
                          size_t n,
                          int *prev_in_word);
```

This function counts how many words exist inside a chunk of data. The `prev_in_word` variable is used to correctly handle cases where a word is split across two different chunks.

Helper Functions in pwordcount.c

```
static void die_perror(const char *msg);
static void write_all(int fd, const void *buf, size_t n);
static size_t read_exact(int fd, void *buf, size_t n);
```

- `die_perror()` prints an error message and exits.
- `write_all()` ensures all bytes are written to the pipe (since `write()` may not send everything at once).
- `read_exact()` safely reads a fixed number of bytes.

5 Word Counting Logic

A word is defined as a sequence of non-whitespace characters. Whitespace is detected using the `isspace()` function.

One challenge was handling words that might be split across two different read operations. For example, if the word “hello” is divided between two chunks, we must count it only once.

To solve this, I used a variable called `prev_in_word` that keeps track of whether the previous chunk ended inside a word.

6 Error Handling

The program includes several safety checks:

- If no filename is provided, it prints a usage message.
- If the file cannot be opened, it prints an error and exits cleanly.
- All system calls such as `pipe()`, `fork()`, `read()`, and `write()` are checked for errors.

- Unused pipe ends are closed properly to prevent deadlocks.

I also made sure that if the parent fails to open the file, the child does not print normal status messages. This avoids confusing output.

7 Separate Compilation

The program is organized into multiple files:

- `pwordcount.c` – main logic and pipe management
- `wordcount.c` – word counting implementation
- `wordcount.h` – function declarations
- `Makefile` – compilation rules

Using separate compilation makes the program easier to understand and maintain.

8 Sample Input and Output

Sample Input File

This is the second project of COMP7500 class.

Program Output

```
$ ./pwordcount input.txt
Process 1 is reading file "input.txt" now ...
Process 1 starts sending data to Process 2 ...
Process 2 finishes receiving data from Process 1 ...
Process 2 is counting words now ...
Process 2 is sending the result back to Process 1 ...
Process 1: The total number of words is 8.
```

9 Design Challenges and Solutions

During the implementation of this project, I encountered several practical design challenges related to process communication and file handling. Below are the main issues I faced and how I addressed them.

9.1 Managing Two Pipes Correctly

The assignment requires two pipes: one to send file data from the parent to the child, and another to send the word count result back.

A common issue when working with pipes is forgetting to close unused file descriptors. If the write-end of a pipe remains open, the reading process will never receive an end-of-file (EOF) signal and may block indefinitely.

Solution: After calling `fork()`, each process immediately closes the pipe ends it does not use. The parent closes the read-end of the first pipe and the write-end of the second pipe. The child does the opposite. This guarantees that EOF is delivered correctly and prevents deadlocks.

9.2 Handling Partial Writes to a Pipe

The `write()` system call does not guarantee that all requested bytes are written in one call. When sending file data through the pipe, this could lead to incomplete transmission if not handled properly.

Solution: I implemented a helper function called `write_all()` that repeatedly calls `write()` until all bytes are sent. This ensures that the entire file content is transmitted to the child process reliably.

9.3 Counting Words Across Buffer Boundaries

The file is read in fixed-size chunks. Because of this, a word may be split between two chunks. If not handled carefully, such words could be counted incorrectly.

Solution: I used a state variable called `prev_in_word` to remember whether the previous buffer ended inside a word. This allows the program to correctly count words even when they span across multiple read operations.

9.4 Clean Behavior When File Cannot Be Opened

If the parent process fails to open the input file, the child process should not continue normal execution or print misleading messages.

Solution: When file opening fails, the parent closes the write-end of the first pipe. This causes the child to receive EOF immediately. The child checks whether it received any data; if not, it exits quietly without printing normal status messages. This results in clean and predictable behavior.

9.5 Ensuring Proper Program Termination

When working with multiple processes, it is important to prevent zombie processes.

Solution: The parent process calls `waitpid()` after finishing its work to ensure the child process terminates properly.